

Humanify — Technical Architecture & Operations

Version: 2.0

Evidence cutoff: 30 July 2026

Audience: Engineering, security, DevOps, technical diligence, enterprise IT.

1. Architecture summary

Humanify is a multi-tenant HRIS SaaS implemented inside a Next.js 15 Pages Router application. Frontend pages, API routes, authentication, scheduled operations, and platform administration are maintained in one repository and the main web/API workload runs as one PM2-managed Next.js process per environment.

Humanify currently favors operational simplicity over microservices:

- One application repository.
- One Next.js process for frontend and APIs.
- Shared PostgreSQL database with `tenant_id`.
- Sequelize and parameterized raw SQL as the canonical write path.
- Prisma retained as read-only/non-canonical for Humanify.
- Nginx and Cloudflare in front of the application.

Extraction into a separate `humanify-core` package or second worker/BFF is explicitly deferred until another deployable requires it.

2. System context

Actors

- Employee, manager, HR administrator, finance/payroll operator.
- Tenant owner.
- Naincode platform operator.
- External IdP, Midtrans, SMTP provider, SumoPod, attendance devices, job portals, and optional object storage.

Application surfaces

- `/humanify/*`: tenant HR operations.
- `/employee/*`: employee ESS.
- `/platform/*`: Naincode control plane.
- `/c/{tenantSlug}/careers`: public tenant careers.
- `/api/humanify/*`: authenticated HRIS APIs.
- `/api/platform/*`: platform control APIs.
- `/api/v1/*`: scoped public enterprise APIs.

UI shells

- Operations: `components/humanify/HumanifyLayout.tsx` over shared `HQLayout`.
- ESS: `components/employee/EmployeePortal.tsx` with separate teal tokens.
- Marketing: `HumanifyMarketingShell`.
- Shared operations tokens: `styles/humanify-tokens.css`.

The different visual surfaces are an intentional ADR, not accidental inconsistency.

3. Runtime topology

Production

- Host directory: `/root/humanify`.
- PM2 process: `humanify`.
- Application port: 3020.
- Database: `humanify`.
- Tenant policy mode: request-bound soft RLS.
- Public domain: `humanify.id`.

Staging

- Host directory: `/root/humanify-staging`.
- PM2 process: `humanify-staging`.
- Application port: 3021.
- Database: `humanify_staging`.
- Tenant policy mode: strict RLS/FORCE.
- Public domain: `staging.humanify.id`.

Infrastructure

- Node.js 20.
- PM2 fork mode, one application instance.
- Nginx reverse proxy.
- Cloudflare edge/TLS.
- PostgreSQL.
- Redis when configured, with memory fallback for selected controls.

Key deployment files:

- `scripts/deploy-humanify-vps.sh`
- `scripts/deploy-humanify-staging-vps.sh`
- `scripts/humanify-ecosystem.config.cjs`
- `scripts/ensure-humanify-crons.sh`

4. Request and API architecture

Normal authenticated request

1. NextAuth resolves JWT/session.
2. `withHQAAuth` attaches the session to the request.
3. Tenant context is established for request-bound database access.
4. Role, permission, module, and plan checks run.
5. The handler executes queries within the expected Sequelize transaction context where request-bound RLS is enabled.
6. Errors are normalized and logged.

Core implementation:

- `lib/middleware/withHQAAuth.ts`
- `lib/permissions/permission-resolver.ts`
- `lib/saas/assert-feature.ts`
- `lib/saas/tenant-request-bound.ts`
- `lib/humanify/api-error.ts`

API boundary rules

- Client-provided `tenantId` is not an authorization source.
- Tenant identity comes from the authenticated session or a verified public integration context.
- Mutating SQL uses `withHQAAuth` or explicit tenant DB context.
- Public webhooks require signatures/secrets and idempotency where applicable.
- Mock HR fallback is prohibited in production.
- `middleware.ts` excludes `/api/*`; every API route must enforce its own authentication or public-token policy.

Static enforcement:

- `npm run lint:humanify-hq-auth`
- `scripts/lint-humanify-hq-auth.js`

5. Authentication and authorization

Authentication

- NextAuth credentials provider.
- SAML handoff/provider path.
- JWT sessions with tenant, role, plan, onboarding, MFA, and impersonation context.
- bcrypt password hashing.
- Hashed invitation and password-reset tokens.
- Login lockout and rate-limit controls.

Primary file:

- `pages/api/auth/[...nextauth].ts`

Page access

`middleware.ts` handles:

- Public route allowlisting.
- Login redirects.
- Onboarding enforcement.
- Tenant MFA enrollment.
- Plan/feature redirects.
- LMS lab gating.
- Tenant career-domain routing.

Authorization

- Role and permission resolution in `withHQAAuth`.
- Feature entitlement checks for pages and APIs.
- Platform routes restricted to `super_admin`, `superadmin`, or `platform_admin`.
- Support impersonation restricted to platform roles and audit logged.
- Enterprise API keys are hashed, scoped, revocable, and require eligible entitlement.

MFA

- RFC6238 TOTP.
- Tenant policy may require enrollment.
- Current infrastructure-error behavior can fail open and should be evaluated against enterprise risk appetite.

6. Multi-tenancy and data isolation

Model

- Shared PostgreSQL database.
- Business rows carry `tenant_id`.
- Session tenant is authoritative.
- Platform operators may use explicit cross-tenant privileges.

Isolation layers

1. Application query scoping (`scopedWhere` and explicit tenant filters).
2. Request-bound PostgreSQL context:
 - `app.current_tenant`
 - `app.is_super_admin`
3. PostgreSQL row-level security policies.
4. Role/permission and plan enforcement.
5. IDOR smoke and weekly scorecard.

Production mode

Production uses application filtering plus request-bound **soft-policy RLS**. Soft means an empty tenant context can be allowed by policy; it does not mean RLS is absent. This makes explicit application scoping critical.

Staging mode

Staging applies strict policies and FORCE RLS. Empty tenant context has been validated to return zero protected employee rows.

Deferred strict production flip

The production flip is manual and gated:

```
CONFIRM_PROD_RLS_STRICT=YES bash scripts/flip-humanify-prod-rls-strict.sh
```

Required evidence includes staging IDOR, job/cron tenant context, restore readiness, and CTO sign-off. See:

- docs/humanify-rls-prod-flip.md
- docs/humanify-rls-strict-staging.md
- docs/humanify-tenant-isolation-faq.md

7. Data architecture

Canonical access path

- Sequelize models and Sequelize transactions.
- Parameterized raw PostgreSQL SQL for complex/reporting paths.
- Runtime connection: lib/sequelize.js.
- Model registry: models/index.js.

Prisma posture

Prisma is not the canonical Humanify write/migration layer. No Humanify write path should be added through Prisma unless the architecture decision changes.

Schema evolution

Current migration posture combines:

- sequelize-cli migrations.
- Humanify-specific migration scripts.
- RLS migration scripts.
- Some runtime CREATE TABLE IF NOT EXISTS safeguards.
- Schema-drift checks through information_schema.
- SAVEPOINT-wrapped optional queries where optional schema must not abort a larger request transaction.

Architectural liabilities

- Runtime table creation weakens deterministic schema ownership.
- Some deploy migration steps tolerate errors.
- The shared model registry includes legacy non-Humanify models.
- Schema drift increases testing and support cost.
- A formal, ordered Humanify migration ledger should replace runtime creation over time.

8. Domain boundaries

Principal Humanify domains:

- SaaS tenancy, plans, billing, signup, invites, partner channel.
- People and organization.
- Attendance, shifts, devices, leave, and overtime.
- Payroll, tax, BPJS, THR, adjustments, audit, and disbursement.
- Claims, travel, private files, and document retention.
- Recruitment, careers, LMS, training, certificates.
- Performance, KPI, analytics, and engagement.
- Lifecycle: onboarding, contracts, assets, offboarding.
- ESS/MSS.
- Enterprise: SSO, API keys, brand controls.
- Platform operations and observability.
- AIMAN/AI assistance.

Reference: docs/humanify-bounded-context.md.

9. Integrations

Midtrans

- Snap checkout.
- Signed notification verification.
- Webhook idempotency.
- Plan activation and billing orders.
- Manual provider fallback when server credentials are missing.
- Partner auto-disbursement is not implemented.

Email

- Verification, reset, invitation, alerts, digests, and payroll messages.
- SMTP/DNS probes and operational checks exist.
- Production token-return fallback behavior during email failure should be reviewed because it can weaken the expected email-only token boundary.

SAML SSO

- Metadata and ACS flow.
- Signed assertion handling.
- Tenant configuration, domain allowlist, and JIT provisioning.
- Synthetic ACS is release-gated.
- Real IdP onboarding is customer-specific.

SumoPod / AIMAN

- OpenAI-compatible provider.
- Enabled only when configured and feature-gated.
- Rule engines remain the source for calculations.
- Write tools require confirmation.

Attendance devices

- ZKTeco HTTP bridge.
- Simulation only outside production.
- Production depends on deployed bridge/device configuration.

Job portals

- Native careers and Google Jobs are strongest.
- Other portal integrations vary from direct integration to outbound/manual readiness and must be described individually.

Document storage

- Private local storage is the default.
- S3/R2 is opt-in.
- Existing local objects are not automatically migrated.

Privy

- Client/webhook code exists.
- UI remains hidden until provider and GA controls are complete.

10. Security architecture

Implemented controls include:

- Password hashing.
- JWT session signing.
- Login lockout and rate limiting.
- TOTP MFA.
- Hashed, expiring invitation/reset tokens.
- Hashed and scoped enterprise API keys.

- Tenant application scoping and RLS.
- Private claim/document storage.
- Session/HMAC file access.
- Webhook signature verification and idempotency.
- Security headers and referrer policy.
- IDOR regression and weekly scorecard.
- Production mock-data guard.
- Support impersonation audit.

Known security risks to track

- `withHQAAuth` error-swallowing around context/entitlement setup can increase reliance on each handler's explicit tenant filter.
- Some scoping helpers can return broad clauses when tenant context is absent.
- Selected public webhook validation paths may fail open when provider secrets are unset.
- MFA and login-guard infrastructure errors currently prioritize availability.
- Some aggregate AI queries rely on request-bound RLS rather than explicit tenant predicates.
- Build configuration skips TypeScript/lint validation; CI filtering is not equivalent to a clean repository build.
- Production strict-policy RLS remains deferred.

These are risk statements, not proof of exploitation. They require tracked remediation, tests, and release evidence.

11. Observability

Current internal observability:

- Structured JSON logs to PM2 stdout.
- In-memory ring buffer.
- Slow-request threshold.
- Persisted errors/warnings in `humanify_obs_events`.
- `/platform/observability`.
- Public shallow/deep health.
- Discord/email alerts and external uptime probes.

Default production direction is internal monitoring. External Sentry.io is deferred by ADR, not accidentally missing.

12. Backup, recovery, and scheduled operations

Documented targets:

- RPO: at most 24 hours.
- RTO: at most 2 hours.
- Default backup retention: 7 days.

Scheduled jobs include:

- Health monitoring.
- Observability and Redis checks.
- Daily database backup.
- Tenant hard-delete processing.
- Document expiry.
- Action digest.
- Weekly IDOR/security scorecard.

See `docs/humanify-backup-restore-runbook.md` and `scripts/ensure-humanify-crons.sh`.

13. Deployment flow

Typical production flow:

1. Validate source and environment.
2. Preserve `.env`, storage, and uploads.
3. Transfer/synchronize application.
4. Install dependencies.
5. Run database migrations/setup scripts.
6. Build Next.js on the VPS.
7. Verify `.next/BUILD_ID` and critical page artifact.
8. Restart PM2 with environment.
9. Probe local and public health.
10. Save PM2 process state.

Known operational risks:

- On-host builds can be slow and fragile.
- Concurrent/orphan `next build` processes have caused recovery work.
- SSH timeout can hide final build status.
- Some migrations and final health checks are non-blocking.
- Single VPS/process is a concentration risk.

Recommended direction:

- Build immutable artifact in CI.
- Promote the same artifact from staging to production.
- Make critical migrations and post-deploy probes blocking.
- Record release ID, migration set, test artifacts, and rollback target.
- Exercise restore/rollback on a schedule.

14. Scalability posture

Current architecture is reasonable for early-stage controlled SaaS but has not established a sustained capacity baseline.

Before horizontal scaling:

- Require shared Redis for distributed rate limits and locks.
- Remove process-memory assumptions.
- Separate scheduled jobs from web request processes.
- Establish connection-pool and database saturation limits.
- Move builds off the production host.
- Define object-storage migration and CDN policy.
- Validate tenant isolation across multiple instances.

15. Technical roadmap

Priority 0

- Close fail-open public webhook/token-delivery behavior.
- Ensure all critical queries fail closed without tenant context.
- Make release evidence and post-deploy verification blocking.
- Reconcile migration ownership and remove runtime schema creation from business requests.

Priority 1

- Immutable CI build/promotion.
- Sustained load, capacity, and Web Vitals baselines.
- Restore/rollback automation.
- Shared Redis requirement for multi-instance deployment.
- Explicit tenant filters in aggregate/All queries even when RLS is active.

Priority 2

- Evaluate production strict RLS only after gates remain green.
- Object-storage migration tooling.
- Split scheduled workers if operational scale requires it.
- Add external observability only when the operational case is justified.

16. Key references

- `components/humanify/HumanifyLayout.tsx`
- `components/employee/EmployeePortal.tsx`
- `pages/api/auth/[...nextauth].ts`
- `lib/middleware/withHQAAuth.ts`
- `lib/saas/tenant-scope.ts`
- `lib/saas/tenant-request-bound.ts`
- `scripts/migrate-humanify-rls.js`
- `scripts/deploy-humanify-vps.sh`
- `scripts/ensure-humanify-crons.sh`
- `docs/humanify-staging-deploy.md`

- docs/humanify-backup-restore-runbook.md
- .hermes/DECISIONS.md